

FIAP GRADUAÇÃO

I Introdução

- ▶ arquivos são fontes importante de armazenamento dados
- ▶ forma universal de troca de informações
- ▶ as operações mais comuns em arquivos são:
 - ▶ ler
 - ▶ escrever
 - ▶ adicionar
- ▶ nos próximos eslaides mostrarei como realizar as 3 operações em arquivos do tipo texto

Leitura de arquivo txt

- ▶ necessário conhecer a localização e o nome dele
- ▶ considerado um recurso como conexões de banco de dados ou cursores
- ▶ também utilizaremos `with` para abrir os arquivos
- ▶ exemplos abaixo:

```
1 with open("dados.txt", "r") as arq:  
2     dados = arq.read()  
3  
4 print(dados)
```

- ▶ o método `read` lê o conteúdo do arquivo inteiro e retorna ele em uma string
- ▶ tome cuidado pois se o arquivo for muito grande pode faltar memória RAM

Leitura de arquivo txt

- ▶ algumas vezes queremos processar os arquivos linha a linha
- ▶ nesse caso, usamos o método `readlines()` que devolve uma lista de strings
- ▶ exemplo abaixo:

```
1 with open("informacoes.txt", "r") as arq:
2     dados = arq.readlines()
3     for linha in dados:
4         print(linha)
```

- ▶ da mesma forma que o `read`, o `readlines` lê o conteúdo do arquivo inteiro em memória

Leitura de arquivo txt

- ▶ para ler linha por linha sem a necessidade de armazenar tudo em memória, temos o método `readline`
- ▶ exemplo abaixo:

```
1  with open("informacoes.txt", "r") as arq:
2      linha = arq.readline()
3      while linha:
4          print(linha)
5          linha = arq.readline()
```

- ▶ quando o arquivo termina, a `linha` recebe uma string **EOF** que indica o término do arquivo e consequentemente a condição é falsa

Resumo dos métodos de leitura de dados

- ▶ `read()` -> `str`: lê o conteúdo inteiro do arquivo retornando como uma String
- ▶ `readlines()` -> `list`: lê o conteúdo inteiro do arquivo retornando como uma lista de strings
- ▶ `readline()` -> `str`: lê o arquivo linha por linha, nesse caso usamos um comando `while` para processar as informações

Gravando dados em um arquivo txt

PROBLEMA Dada uma lista de times de futebol do campeonato brasileiro de 2023, simule os resultados desse campeonato. Não se esqueça que temos turno e retorno.

```
1 import random
2
3 times = ["Palmeiras", "Botafogo", "Flamengo", "Corinthians",
4          "Santos", "Internacional", ...]
5
6 with open("resultados.txt", "w") as arq:
7     for i in range(len(times)):
8         for j in range(i+1, len(times)):
9             gc = random.randint(0, 7)
10            gv = random.randint(0, 7)
11            arq.write(f"{times[i]} {gc} X {gv} {times[j]}\n")
12
13     for i in range(len(times)):
14         for j in range(i+1, len(times)):
15             gc = random.randint(0, 7)
16             gv = random.randint(0, 7)
17             arq.write(f"{times[j]} {gc} X {gv} {times[i]}\n")
```

Adicionando dados em um arquivo txt

- ▶ para abrir um arquivo para leitura, usamos `open("arquivo", "r")`
- ▶ para criar um novo arquivo, usamos `open("arquivo", "w")`
- ▶ para adicionar dados em um arquivo existente, usamos `open("arquivo", "a")`
- ▶ onde "a" vem de append, veja abaixo um exemplo:

```
1 with open("resultados.txt", "a") as arq:  
2     arq.write('Finalizando as partidas')
```


Exercícios

1. Suponha que temos 3 lojas que comercializa alguns produtos: celular, smart tv, notebook, geladeira e máquina de lavar. Sua tarefa é gerar um arquivo txt contendo o faturamento diário de 2023 das três lojas. Use a biblioteca random para isso. O arquivo deverá ter o seguinte formato:

`<produto>;<marca>;<loja>;<data>;<qtd>;<valor unitário>`

Use o ; como separador dos campos e o seu arquivo precisa conter, pelo menos, 5 mil entradas. Como sugestão, utilize listas e dicionários para representar as lojas, os produtos e suas respectivas marcas (use 2 para cada produto) e os respectivos valores para elas. Creio que dessa forma ficará mais fácil para criar os registros a serem gravados nos arquivos.

Exercícios

2. Faça um programa que lê o arquivo criado no exercício 1 e grava o conteúdo desse arquivo em uma tabela do banco de dados chamada de faturamento. Caso queira, aplique as regras de normalização de banco de dados, ou seja, crie mais tabelas para seu modelo estar na 3ª forma normal. É provável que a inserção de tantos registros fique lenta, leia a documentação do oracledb na seção **Executing Batch Statements and Bulk Loading**.
3. Escreva um programa ou duas funções para gerar dois relatórios em arquivos txt; o primeiro deverá ter as seguintes informações:
`<loja>;<mês>;<faturamento>`
e o segundo:
`<produto>;<mês>;<faturamento>`
Lembre-se que faturamento é o produto da quantidade pelo valor unitário do produto.

Exercícios

4. Faça uma consulta em uma tabela de banco de dados e grave o resultado em um arquivo.
5. Escreva um programa que gera um arquivo contendo vários inserts de banco de dados, imagine que você precisa popular uma tabela de forma automática. Use a biblioteca Faker para esse propósito.